The FamilySearch GEDCOM Specification

7.0.16

Prepared by the

Family History Department
The Church of Jesus Christ of Latter-day Saints

18 March 2025

Suggestions and Correspondence:

Family History Department
15 East South Temple Street
Salt Lake City, UT 84150 USA

GEDCOM@FamilySearch.org

Copyright 1984–2025 Intellectual Reserve, Inc. All rights reserved. A service provided by The Church of Jesus Christ of Latter-day Saints.

Licensed under the Apache License, Version 2.0 (the “License”); you may not use this file except in compliance with the License. You may obtain a copy of the License at

> http://www.apache.org/licenses/LICENSE-2.0

Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an “AS IS” BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

In accordance with the Apache 2.0 license that governs this work, any other work that is based on or derived from this work must include a readable copy of the following NOTICE. For more information, please refer to the full copy of the Apache 2.0 license.

> NOTICE:
>
> This work comprises, is based on, or is derived from the FAMILYSEARCH GEDCOM™ Specification, © 1984-2025 Intellectual Reserve, Inc. All rights reserved.
>
> “FAMILYSEARCH GEDCOM™” and “FAMILYSEARCH®” are trademarks of Intellectual Reserve, Inc. and may not be used except as allowed by the Apache 2.0 license that governs this work or as expressly authorized in writing and in advance by Intellectual Reserve, Inc.

Contents

- Introduction 6
  - Purpose and Content of *The FamilySearch GEDCOM Specification* 6
  - Purpose for Version 7.x 7
  - A Guide to Version Numbers 7
  - URIs and Prefix Notation 8
- 1 Hierarchical container format 9
  - 1.1 Characters 9
  - 1.2 Structures 10
  - 1.3 Lines 11
  - 1.4 The Header and Trailer 15
  - 1.5 Extensions 16
    - 1.5.1 Extension Tags 19
    - 1.5.2 Requirements and Recommendations 21
    - 1.5.3 Extension versus Standard 22
  - 1.6 Removing data 23
- 2 Data types 24
  - 2.1 Text 24
  - 2.2 Integer 24
  - 2.3 Enumeration 24
  - 2.4 Date 25
  - 2.5 Time 27
  - 2.6 Age 28
  - 2.7 List 29
  - 2.8 Personal Name 30
  - 2.9 Language 30
  - 2.10 Media Type 30
  - 2.11 Special 31
  - 2.12 File Path 31
- 3 Genealogical structures 33
  - 3.1 A Metasyntax for Structure Organization 33
  - 3.2 Structure Organization 35
    - 3.2.1 Document 35
    - 3.2.2 Records 37
    - 3.2.3 Substructures 43
  - 3.3 Structure Meaning 59
    - 3.3.1 Events 59
    - 3.3.2 Attributes 62
    - 3.3.3 Latter-day Saint Ordinances 64

- 3.3.4 Structure types 65
- 3.4 Enumeration Values 92
  - `g7:enumset-ADOP` 92
  - `g7:enumset-EVEN` 92
  - `g7:enumset-EVENATTR` 93
  - `g7:enumset-MEDI` 93
  - `g7:enumset-PEDI` 94
  - `g7:enumset-QUAY` 94
  - `g7:enumset-RESN` 94
  - `g7:enumset-ROLE` 95
  - `g7:enumset-SEX` 96
  - `g7:enumset-FAMC-STAT` 96
  - `g7:enumset-ord-STAT` 97
  - `g7:enumset-NAME-TYPE` 98
- 4 The FamilySearch GEDZIP file format 100
- 5 Contributors 101
- 6 Appendix A: Known Calendars and Dates 103
  - 6.1 Known Calendars 103
    - `GREGORIAN` 103
    - `JULIAN` 104
    - `FRENCH_R` 104
    - `HEBREW` 105
  - 6.2 Dual dates 106
  - 6.3 Calendars in date ranges and date periods 106

Introduction

FamilySearch GEDCOM 7.0 was released in 2021 as the latest version of the GEDCOM format for the transmission and storage of genealogical information. GEDCOM was developed by the Family History Department of The Church of Jesus Christ of Latter-day Saints to provide a flexible, uniform format for exchanging computerized genealogical data. Its first purpose is to foster sharing genealogical information and to develop a wide range of inter-operable software products to assist genealogists, historians, and other researchers. Its second purpose is as a long-term storage format for preserving genealogical information in an open, standard format that will be accessible and understood by future genealogists and the systems they use.

"GEDCOM" is an acronym for **GE**nealogical **D**ata **COM**munication and is traditionally pronounced "ˈdʒɛdkɑm."

Purpose and Content of *The FamilySearch GEDCOM Specification*

The FamilySearch GEDCOM Specification is a technical document written for computer programmers, system developers, and technology-aware users. This document describes a document and file format as follows:

- A hierarchical container format (see Chapter 1 (p.9))
- A set of data types (see Chapter 2 (p.24))
- A set of genealogical structures (see Chapter 3 (p.33))
- The FamilySearch GEDZIP file format (see Chapter 4 (p.100))

Chapter 1 describes a hierarchical container format. This container format is a general-purpose data representation language for representing any kind of structured information in a sequential medium, similar to XML, JSON, YAML, or SDLang. Chapter 1 discusses the syntax and identification of structured information in general, but it does not deal with the semantic content of any particular kind of data.

Chapter 2 describes several data types used to represent genealogical information, such as a date format that permits dating in multiple calendar systems.

Chapter 3 describes a set of nested genealogical structures for representing historical claims, such as individuals, families, and events; sourcing information, such as sources, repositories, and citations; and research metadata, such as information about researchers and rights.

A set of structures conforming to the first 3 chapters of *The FamilySearch GEDCOM Specification* is called a FamilySearch GEDCOM dataset. A string of octets encoding a dataset is called a data stream.

Chapter 4 describes a file format for bundling a dataset with a set of media files or other supporting documents.

> *Note* — Prior to 7.0:
>
> - The container format was called “the GEDCOM data format.”
> - The data types were unnamed and described in various places throughout the document.
> - The genealogical structures were known as “the Lineage-Linked GEDCOM Form.”

In addition to this specification itself, additional supplimental material can be found at https://gedcom.io, including:

- Links to example and supporting software
- Frequently asked questions for users and developers
- A guide to migrating from 5.5.1 to 7.0
- Ways to engage in the community

Purpose for Version 7.x

There have been multiple prior releases of this specification, with somewhat idiosyncratic version numbering. The first public comment draft was released in 1984. The previous major version was 5.5.1 which was released in draft status in November 1999 and re-released as a standard in October 2019.

Version 7.0 has a number of goals, including

- Clarify ambiguities in the specification.
- Simplify implementations by removing special-case handling.
- Modernize character encoding, length restrictions, and specification wording.
- Introduce semantic versioning (see https://semver.org/).
- Add better multimedia handling, negative assertions, and notes with markup.
- Add support for common extensions to 5.5.1.
- Provide tools for better interoperability of extensions.

Version 7.0 introduces several breaking changes with version 5.5.1; 5.5.1 files are, in general, not valid 7.0 files and *vice versa*. These breaking changes were necessary to remove complicated constructs left over from earlier versions. For a complete list of changes, see the accompanying changelog.

A Guide to Version Numbers

Starting with version 7.0.0, version numbers use semantic versioning. The 3 numbers are titled *major.minor.patch*.

A new *major* version may make arbitrary changes to the specification. Distinct major versions are not in general either forward or backward compatible with one another.

A new *minor* version will preserve the validity of data from all previous minor versions. It may make additional data valid, for example by adding new structure types, allowing current structures in new contexts, or adding new enumerated values or calendars. A minor release will not change the semantic meaning of data from previous minor releases, so for example a 7.0 document is also a valid 7.1 document and represents the same information in both.

A new *patch* version is a clarified or improved specification for the same data and introduces no changes in the data itself. Any software that correctly implements *X.Y.Z* also correctly implements *X.Y.W*. If there is an ambiguity or contradiction in the specification, it will be resolved in a patch version unless it is known that implementations interpreted the spec differently and that clarifying the intended meaning would cause incompatibilities between those implementations.

It is recommended that implementations accept all data at their own or a lesser minor version, regardless of the patch version. It is also recommended that they import data from subsequent minor versions by treating any unexpected structures, enumerations, or calendars as if they were extensions (p.16).

URIs and Prefix Notation

This document defines Uniform Resource Identifiers (URIs) to unambiguously identify various concepts, including structure types, data types, calendars, enumerated values, and so on. In a few places, existing URIs defined by other bodies are used, following the best practice that a new URI should not be introduced for a concept for which a URI is known.

Rather than write out URIs in full, we use prefix notation: any URI beginning with 1 of the following short prefixes followed by a colon is shorthand for a URI beginning with the corresponding URI prefix

Short Prefix	URI Prefix
`g7`	`https://gedcom.io/terms/v7/`
`xsd`	`http://www.w3.org/2001/XMLSchema#`
`dcat`	`http://www.w3.org/ns/dcat#`

> *Example* — When the specification says `xsd:string`, it means `http://www.w3.org/2001/XMLSchema#string`. This is a specification shorthand only; the string "`xsd:string`" is not the URI defining this concept, `http://www.w3.org/2001/XMLSchema#string` is.

1. Hierarchical container format

1.1. Characters

Each data stream is a sequence of octets or bytes. The octets encode a sequence of characters according to the UTF-8 character encoding as described in §10.2 of ISO/IEC 10646:2020 and in RFC 3629.

> *Note* — Previous versions allowed multiple character encodings, defaulting to ANSEL. 7.0 only uses the UTF-8 character encoding.

A file containing a FamilySearch GEDCOM data stream should use the filename extension `.ged`.

The first character in each data stream should be U+FEFF, the byte-order mark. If present, this initial character has no meaning within this specification but serves to indicate to other systems that the file uses the UTF-8 character encoding.

Certain characters must not appear anywhere within a data stream:

- The C0 control characters other than tab and line endings (U+0000–U+001F except U+0009, U+000A and U+000D)
- The DEL character (U+007F)
- Surrogates (U+D800–U+DFFF)
- Invalid code points (U+FFFE and U+FFFF)

Implementations should be aware that bytes per character and characters per glyph are both variable when using UTF-8. Use of Unicode-aware processing and display libraries is recommended.

Character-level grammars are specified in this document using Augmented Backus-Naur Form (ABNF) as defined in STD 68 and modified in RFC 7405. We use the term “production” to refer to an ABNF rule, supported by any other rules it references.

> *Note* — The following is a brief summary of the parts of ABNF, as defined by STD 68 and RFC 7405, that are used in this document:
>
> - A rule consists of a rulename, an equals sign `=`, and 1 or more alternative matches.
> - Alternatives are separated by slashes `/`.
> - The first line of a rule must not be indented; the second and subsequent lines of a rule must be indented.
> - Comments are introduced with a semi-colon `;`.
> - Unicode codepoints are given in hexadecimal preceded by `%x`. Ranges of allowed codepoints are given with a hyphen `-`.
> - Double quotes delimit literal strings. Literal strings are case-insensitive unless they are preceded by `%s`.
> - Parentheses `()` group elements. Brackets `[]` mark optional content. Preceding a group or element by `*` means any number may be included. Preceding a group or element by `1*` means 1 or more may be included.

The banned characters can be expressed in ABNF as production `banned` (p.9):

```
banned = %x00-08 / %x0B-0C / %x0E-1F ; C0 other than LF CR and Tab
       / %x7F                        ; DEL
       / %x80-9F                     ; C1
       / %xD800-DFFF                 ; Surrogates
       / %xFFFE-FFFF                 ; invalid
; All other rules assume the absence of any banned characters
```

All other ABNF expressions in this document assume the absence of any characters matching production `banned`.

This document additionally makes use of the following named character sets in ABNF:

```
; DIGIT     = %x30-39   ; 0 through 9 -- defined in RFC 5234 section B.1
nonzero     = %x31-39   ; 1 through 9
ucletter    = %x41-5A   ; A through Z
underscore  = %x5F      ; _
atsign      = %x40      ; @
```

1.2. Structures

A **structure** consists of a structure type, an optional **payload**, and a collection of substructures. The payload is a value expressed as a string using 1 of several data types, as described in Chapter 2 (p.24).

Every structure is either a **record**, meaning it is not contained in any other structure's collection of substructures, or it is a **substructure** of exactly 1 other structure. The other structure is called its **superstructure**. Each substructure either refines the meaning of its superstructure, provides metadata about its superstructure, or introduces new data that is closely related to its superstructure.

Each **structure type** is identified by a URI and defines several properties of any structure with that type, including

- The meaning of structures of this type.
- The payload type of the structure's payload, which shall be one of
  - no payload, or
  - a pointer to a record with a specific structure type, or
  - a data type (p.24); if an enumeration (p.24) or list of enumerations (p.29), also a set of permitted enumeration values.
- Which structure types may appear as substructures of the structure and with what **cardinality** they may appear. Cardinality is specified by two flags:
  - whether a substructure of this type is required or not; and
  - whether multiple substructures of this type are permitted or not.

The collection of substructures is partially ordered. Substructures with the same structure type are in a fixed order, but substructures with different structure types may be reordered. The order of substructures of a single type indicates user preference, with the first substructure being the most-preferred value, unless a different meaning is explicitly indicated in the structure's definition.

A structure must have either a non-empty payload or at least 1 substructure. Empty payloads and missing payloads are considered equivalent. The remainder of this document uses "payload" as shorthand for "non-empty payload".

> *Note* — Unlike structures, pseudo-structures needn't have either payloads or substructures. `TRLR` (p.90) never has either, and `CONT` (p.68) doesn't when payloads contain empty lines.

A structure is a representation of data about its **subject**. Examples include the entity, event, claim, or activity that the structure describes.

Datasets also contain 3 types of pseudo-structures:

- The header resembles a record, comes first in each document, and contains metadata about the entire document in its substructures. See The Header (p.15) for more details.
- The trailer resembles a record, comes last in each document, and cannot contain substructures.
- A line continuation resembles a substructure, comes before any other substructures, is used to encode multi-line payloads, and cannot contain substructures.

Previous versions limited the number of characters that could appear in a structure, record, and payload. Those restrictions were removed in 7.0.

1.3. Lines

A **line** is a string representation of (part of) a *structure*. A line consists of a level, optional cross-reference identifier, tag, optional line value, and line terminator. It matches the production `Line`:

```
Line    = Level D [Xref D] Tag [D LineVal] EOL

Level   = "0" / nonzero *DIGIT
D       = %x20                            ; space
Xref    = atsign 1*tagchar atsign         ; but not "@VOID@"
Tag     = stdTag / extTag
LineVal = pointer / lineStr
EOL     = %x0D [%x0A] / %x0A              ; CR-LF, CR, or LF

stdTag  = ucletter *tagchar
extTag  = underscore 1*tagchar
tagchar = ucletter / DIGIT / underscore

pointer = voidPtr / Xref
voidPtr = %s"@VOID@"

nonAt   = %x09 / %x20-3F / %x41-10FFFF    ; non-EOL, non-@
nonEOL  = %x09 / %x20-10FFFF              ; non-EOL
lineStr = (nonAt / atsign atsign) *nonEOL ; leading @ doubled
```

The **level** matches production `Level` (p.11) and is used to encode substructure relationships. Any line with level 0 encodes a record or a record-like pseudo-structure. Any line with level $x > 0$ encodes a substructure of the structure encoded by the nearest preceding line with level $x - 1$.

> *Note* — Previous versions allowed spaces and blank lines to precede the level of a line. That permission was removed from 7.0 to simplify parsing.

The **cross-reference identifier** matches production `Xref` (but not `voidPtr`) and indicates that this is a structure to which pointer-type payloads may point. Each cross-reference identifier must be unique within a given data document. Cross-reference identifiers are not retained between data streams and should not be made visible to the user to avoid them referring to transient data within notes or other durable data.

Each record to which other structures point must have a cross-reference identifier. A record to which no structures point may have a cross-reference identifier, but does not need to have one. A substructure or pseudo-structure must not have a cross-reference identifier.

The **tag** matches production `Tag` and encodes the structure's type. Tags that match the production `stdTag` are defined in this document. Tags that match `extTag` are defined according to Extensions (p.16).

The same tag may be used to represent multiple structure types. The structure type of each structure is identified by its tag and the type of its superstructure. The mapping between (superstructure type, tag) pairs and structure types is given elsewhere in this document (for standard structure types and tags) or the [schema] and extension authors' documentation (for extension structure types and tags).

> *Example* — The tag `ADOP` is used in this document to represent two structure types. Which one is meant can be identified by the superstructure type as follows:
>
> | Superstructure type | Structure type identified by tag `ADOP` |
> |---|---|
> | `g7:record-INDI` (p. 75) | `g7:ADOP` |
> | `g7:ADOP-FAMC` (p.73) | `g7:FAMC-ADOP` |
>
> An extension-defined substructure (p.16) could also be used to place either of these structure types in extension superstructures.
>
> The `ADOP` tag is also used in the set of enumerated values permitted by the `g7:DATA-EVEN` (p.72), `g7:SOUR-EVEN` (p.72), and `g7:NO` (p.79) structure types.

The **line value** matches production `LineVal` (p.11) and encodes the structure's payload. Line value content is sufficient to distinguish between pointers and line strings. Pointers are encoded as the cross-reference identifier of the pointed-to structure. Each non-pointer payload may be encoded in 1 or more line strings (line continuations encode multi-line payloads in several line strings). The exact encoding of non-pointer payloads is dependent on the data type of the payload, as determined by the structure type. The data type of non-pointer payloads cannot be fully determined by line value content alone.

Note that production `LineVal` does not match the empty string. Because empty payloads and missing payloads are considered equivalent, both a structure with no payload and a structure with the empty string as its payload are encoded with no `LineVal` and no space after the `Tag`.

> *Example* — The payload of a `MARR` structure has type `[Y|<NULL>]`, which is optional but if present cannot be the empty string. The payload of a `EVEN` structure has type `Text` (p.24), which is not optional but can be the empty string. The `Line` encoding a no-payload `MARR` (p. 77) is "`1 MARR`" and the `Line` encoding an empty-payload `EVEN` is "`1 EVEN`"; both `Line`s have no `LineVal` and no trailing space.

If a line value matches production `Xref`, the same value must occur as the cross-reference identifier of a structure within the document. The special `voidPtr` production is provided to encode null pointers.

If the first character of the string stored in a line string is U+0040 (`@`), the line string must escape that character by doubling that `@`.

> *Note* — Previous versions required doubling all `@` in a line value, but such doubling was not widely implemented in practice. `@` is only doubled in this version if it is the first character of a line string.

> *Example* — A structure with tag `NOTE` (p.80), level 1, and a 2-line payload where the first line is "`me@example.com is my email`" and the second line is "`@me and @I are my social media handles`" would be encoded as
>
> ```
> 1 NOTE me@example.com is my email
> 2 CONT @@me and @I are my social media handles
> ```

> *Note* — Line values that match neither `Xref` nor `lineStr` are prohibited. They have been used in previous versions (for example, a line value beginning `@#D` was a date in versions 4.0 through 5.5.1) and may be used again in a future version if an appropriate need arises.

The components of a line are each separated by a single **delimiter** matching production `D`. A delimiter is always a single space character (U+0020). Using multiple delimiters between components of a line is prohibited. Thus if the tag is followed by 2 spaces, the first space is a delimiter and the second space is part of the line value.

All characters in a payload must be preserved in the corresponding line value, including preserving any leading or trailing spaces.

Each line is ended by a **line terminator** matching production `EOL`. A line terminator may be a carriage return U+000D, line feed U+000A, or a carriage return followed by a line feed. The same line terminator should be used on every line of a given document.

Line values cannot contain internal line terminators, but some payloads can. If a payload contains a line terminator, the payload is split on the line terminators into several payloads. The first of these split payloads is encoded as the line value of the structure's line, and each subsequent split payload is encoded as the line value of a **line continuation** pseudo-structure placed immediately following, and with one greater level than, the structure's line. The tag of a line continuation pseudo-structure is `CONT` (p. 68). The order of the line continuation pseudo-structures matches the order of the lines of text in the payload.

> *Note* — Versions prior to 7.0 had another `CONT`-like tag, `CONC`, which split line values without introducing a line break. `CONC` does not appear in version 7. To support multi-version GEDCOM parsers, the `CONC` tag is reserved and will not appear as the tag of a structure type.

Line continuation pseudo-structures are not considered to be structures. While they match production `Line` and their level and position makes them appear to be substructures of the structure, they are actually a continuation of the encoding of the structure's payload and are not part of a structure's collection of substructures. They must appear immediately following the line whose payload they are encoding and before any other line.

Because line terminators in payloads are encoded using line continuations, it is not possible to distinguish between U+000D and U+000A in payloads.

> *Note* — Previous versions limited the number of characters that could appear in a tag, cross-reference identifier, and line-value. Those restrictions were removed in version 7.0. The `CONC` pseudo-structure, which allowed line values to have a shorter length restriction than payloads, was also removed.

> *Example* — The following are examples of valid but unrelated lines:
>
> - level 0, cross-reference identifier `@I1234@`, tag `INDI`, no line value.
>
> ```
> 0 @I1234@ INDI
> ```
>
> - level 1, no cross-reference identifier, tag `CHIL` (p.68), pointer line value pointing to the structure with cross-reference identifier "`@I1234@`".
>
> ```
> 1 CHIL @I1234@
> ```
>
> - level 1, no cross-reference identifier, tag `NOTE` (p.80), and line value + continuation pseudo-structure to encode a 4-line payload string: "`This is a note field that`", "`  spans four lines.`", "", and "`(the third line was blank)`". Note that leading and trailing spaces are preserved.
>
> ```
> 1 NOTE This is a note field that
> 2 CONT   spans four lines.
> 2 CONT
> 2 CONT (the third line was blank)
> ```

1.4. The Header and Trailer

Every dataset must begin with a header pseudo-structure and end with a trailer pseudo-structure.

The trailer pseudo-structure has level `0`, tag `TRLR` (p.90) and no line value or substructures. The trailer has no semantic meaning; it is present only to mark the end of the dataset.

The header pseudo-structure has level `0`, tag `HEAD` (p.74), and no line value. The substructures of the header pseudo-structure provide metadata about the entire dataset. Some of those substructures are defined here; others are defined in Chapter 3 (p.33) or by extensions.

Every header must contain a substructure with a known tag that identifies the specification to which the dataset complies. For FamilySearch GEDCOM 7.0, this is the `GEDC` (p.74) structure described in Chapter 3 (p.74).

A header should contain an extension schema structure with tag `SCHMA` (p.85) as described in Extensions (p.16).

1.5. Extensions

A **standard structure** is a structure whose type, tag, meaning, superstructure, and cardinality within the superstructure are described in this document. This includes records such as `INDI` and substructures such as `INDI.NAME` (p.79).

The recommended way to go beyond the set of standard structure types in this specification or to expand their usage is to submit a feature request on the FamilySearch GEDCOM development page so that the ramifications of the proposed addition and its interplay with other proposals may be discussed and the addition may be included in a subsequent version of this specification.

This specification also provides multiple ways for extension authors to go beyond the specification without submitting a feature request, which are described in the remainder of this section.

Extensions can introduce new structure types, new enumeration values, new calendars with their associated months, and new data types. They can also extend existing structures with new permitted substructure types and extend existing enumeration-type payloads with new permitted values. Extensions cannot change existing meanings, cardinalities, or calendars.

A **tagged extension structure** is a structure whose tag matches production `extTag` (p.11). Tagged extension structures may appear as records or substructures of any other structure. Their meaning is defined by their tag, as is discussed more fully in the section Extension Tags (p.19).

Any substructure of a tagged extension structure that uses a tag matching `stdTag` is an **extension-defined substructure**. Substructures of an extension-defined substructure that uses a tag matching `stdTag` are also extension-defined substructures, but this specification deprecates using a `stdTag` with a definition that does not match any standard type with that tag. The meaning and use of each extension-defined substructure is defined by the tagged extension structure it occurs within, not by its tag alone nor by this specification.

Example — In the following

```
0 @P1@ _LOC
1 NAME Βυζάντιον
2 DATE FROM 667 BCE TO 324
1 _POP 15149358
2 DATE 31 DEC 2020
0 @I1@ INDI
1 BIRT
2 _LOC @P1@
```

- Both uses of `_LOC` are tagged extension structures, as is `_POP`.
- `_LOC.NAME` and `_LOC.NAME.DATE` are both extension-defined substructures. Their meaning is defined by the specification defining `_LOC`, but since no standard definition of `NAME` (p.79) permits `DATE` (p.69) as a substructure, such use is deprecated.
- `_POP.DATE` is an extension-defined substructure. Its meaning is defined by the specification defining `_POP`.
- Even though both `DATE`s appear to have `g7:type-DATE` payloads, we can't know that is the intended data type without consulting the defining specifications of `_LOC` and `_POP`, respectively. The first might be a `g7:type-DATE#period` and the second a `g7:type-DATE#exact`, for example.

If an extension-defined substructure has a tag that is also used by one or more standard structures, its meaning and payload type should match at least one of those standard structure types.

Example — An extension-defined substructure with tag "`DATE`" should provide a date or date period relevant to its superstructure, as do all `DATE`-tagged structures in this specification. Extensions should not use "`DATE`" to tag a structure describing anything else (even something that might reasonably be abbreviated "date", such as someone an individual dated).

As a special case, a tagged extension structure can be defined to have a standard structure type. These are called **relocated standard structures** and can only appear with superstructures that are not documented as a superstructure of that structure type in this specification. The extension-defined substructures of a relocated standard structure are the substructure types documented in this specification for that structure type, including usual limitations on cardinality, payloads, substructures, etc.

> *Example* — Suppose `_DATE` is defined to mean a `g7:DATE` (using a documented extension tag (p.19)). Then in the following
>
> ```
> 0 @I1@ INDI
> 1 NAME John /Doe/
> 2 _DATE FROM 6 APR 1917 TO 11 NOV 1918
> 3 PHRASE During America's involvement in the Great War
> 1 BIRT
> 2 PLAC Queens, New York, New York, USA
> ```
>
> - `_DATE` is a relocated standard structure with type `g7:DATE`, with the usual payload type and meaning of a `g7:DATE`.
> - `PHRASE` is the structure type expected with that tag as a substructure of `g7:DATE`: namely, `g7:PHRASE` (p.81).
> - `_DATE` can not be used as a substructure of `BIRT` (p.67) because `BIRT` has a documented `g7:DATE` substructure with tag `DATE`.
> - `BIRT` can not be used as a substructure of `_DATE` or `_DATE.PHRASE` because neither structure type has a documented substructure with tag `BIRT`.

Because all substructures have meanings defined relative to their superstructures and no records do, standard records cannot be relocated and relocated standard structures cannot be used as records.

> *Example* — The `g7:PLAC` (p.82) substructure documents where the event described in its superstructure occurred. If an application wants a record to describe the place itself it should create a new URI for that extension; reusing `g7:PLAC` for a record with no superstructure is not appropriate.

All other non-standard structures are prohibited. Examples of prohibited structures include, but are not limited to,

- a record or substructure of a standard structure using a tag matching production `stdTag` (p.11) that is not defined in this document;
- any substructure with cardinality `{0:1}` appearing more than once;
- a standard substructure appearing as a record or vice-versa;
- a standard structure whose payload does not match the requirements of this document.

> *Note* — In some cases, an extension may need to allow multiple structures where this document allows only 1. The recommended way to do this is to create an extension tag and URI and serve a page describing how the semantics of the structure have been extended to allow multiple instances.
>
> > *Example* — Suppose I have multiple sources that give different ages of the wife at a wedding; however, this specification allows only 1 `MARR` (p.61).`WIFE`.`AGE`. An extension could not include multiple `MARR`.`WIFE` nor `MARR`.`WIFE`.`AGE`, but could define a new extension `_AGE`, give it a URL, and provide the following definition of this extension structure type at that URL:
> >
> > > Alternate age: an age attested by some source, but not accepted by the researcher as the actual age of the individual. If the age is accepted by the researcher, the standard tag `AGE` (p.66) should be used instead.
> >
> > This alternate age extension structure could be used as follows:
> >
> > ```
> > 1 MARR
> > 2 WIFE
> > 3 AGE 27y
> > 3 _AGE 22y
> > ```

Enumerated values may be extended with new values that match production `extTag` (p.11). Enumerations may not use standard values from other enumeration sets.

> *Example* — The following is not allowed because `PARENT` is defined as a value for `ROLE` (p.84), not for `RESN` (p.83)
>
> ```
> 0 @BAD@ INDI
> 1 RESN PARENT
> 1 NOTE The above enumeration value is not allowed
> ```

Dates may be extended provided they use a calendar that matches production `extTag`. Dates with extension calendars may also use extension months and epochs.

1.5.1. Extension Tags

Each use of the `extTag` production is called an extension tag, including when used as a tag, calendar, month, epoch, or enumerated value. Each `extTag` is either a *documented extension tag* or an *undocumented extension tag*. It is recommended that documented extension tags be used instead of undocumented extension tags wherever possible.

A **documented extension tag** is a tag that is mapped to a URI using the schema structure. The schema structure is a substructure of the header with tag `SCHMA` (p.85). It should appear within the document before any extension tags. The schema's substructures are tag definitions.

A tag definition is a structure with tag `TAG` (p.86). Its payload is an extension tag, a space, and a URI and defines that extension tag to be an abbreviation for that URI within the current document.

Example — The following header

```
0 HEAD
1 SCHMA
2 TAG _SKYPEID http://xmlns.com/foaf/0.1/skypeID
2 TAG _MEMBER http://xmlns.com/foaf/0.1/member
```

defines the following tags

Tag	Means
`_SKYPEID`	`http://xmlns.com/foaf/0.1/skypeID`
`_MEMBER`	`http://xmlns.com/foaf/0.1/member`

Note that at the time of writing, the FOAF URIs used in this example are not URLs.

The meaning of a documented extension tag is identified by its URI, not its tag. As such each documented extension tag needs its own URI: it is its URI, not its tag, that defines its meaning. Documented extension tags can be changed freely by modifying the schema, though it is recommended that documented extension tags not be changed. However, a tag change may be necessary if a product picks the same tags for URIs that another product uses for different URIs. A given schema should map only one tag to each URI.

Example — The following 2 document fragments are semantically equivalent and a system importing one may export it as the other without change of meaning.

```
0 HEAD
1 SCHMA
2 TAG _SKYPEID http://xmlns.com/foaf/0.1/skypeID
0 @I0@ INDI
1 _SKYPEID example.person
```

```
0 HEAD
1 SCHMA
2 TAG _SI http://xmlns.com/foaf/0.1/skypeID
0 @I0@ INDI
1 _SI example.person
```

It is recommended that the URIs used for documented extension tags be URLs that can be used to access documentation for the meaning of the tag.